



Prof. Gerry Tsoukalas, Prof. Mohannad Elhamod

Boston University Questrom School of Business

Why Evaluate?

- Objective is to assess the **quality** and **effectiveness** of a given model
 - Model deployment requires significant capital
 - Evaluation is required before deployment
 - Invalid models represent a big waste – they can lead companies astray
- Multiple models can be used to classify or predict
 - Each model usually has multiple possible settings (also called hyperparameters)
 - To choose the best model, we need ways to compare each model's performance

Difficulty: Multiple Possible Evaluation Criteria

- We generally focus on the model's ability to predict the **right value (regression) or class (classification)**.
- Other factors are also important:
 - Speed (Computational cost) of model construction and inference.
 - Robustness to noise
 - Interpretability

Model Generalization

- Example:



<https://medium.com/>

- [Example 2](#)
- When need to make sure our model makes good predictions AND for good the right reasons!

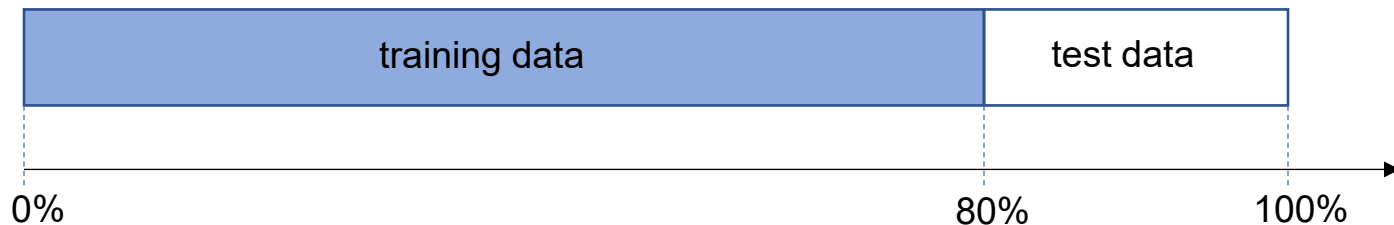
First step: Training / Testing Split

- Objective: partition the data into training and testing parts
 - In-sample (train data) evaluation: how well does my model 'fit' the data
 - Out-of-sample (test data) evaluation: model predictive power on new data
- Example 'diabetes.csv' dataset:

	X									y
	pregnancies	glucose	diastolic	triceps	insulin	bmi	dpf	age	diabetes	
0	6	148	72	35	0	33.6	0.627	50	1	
1	1	85	66	29	0	26.6	0.351	31	0	
2	8	183	64	0	0	23.3	0.672	32	1	
3	1	89	66	23	94	28.1	0.167	21	0	
4	0	137	40	35	168	43.1	2.288	33	1	

Method #1: Hold-out

- Split your data set into two parts
 - training set: ~80% of your **rows**
 - test set: ~20% of remaining **rows**
 - Other common splits: 2/3 -1/3, 50/50 (depends on nature of dataset)
- tweaks: random sampling, repeated hold-out

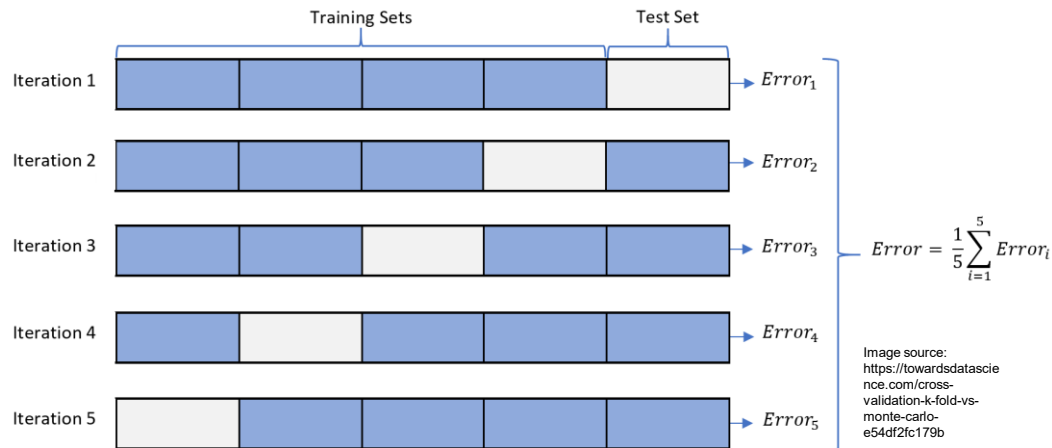


Python package:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Method #2: k-fold cross-validation (cv)

- randomly divide the data set into k groups (example with k = 5 below)
- use k-1 groups to train model and 1 group to test model
- repeat k times, and take average of error estimates



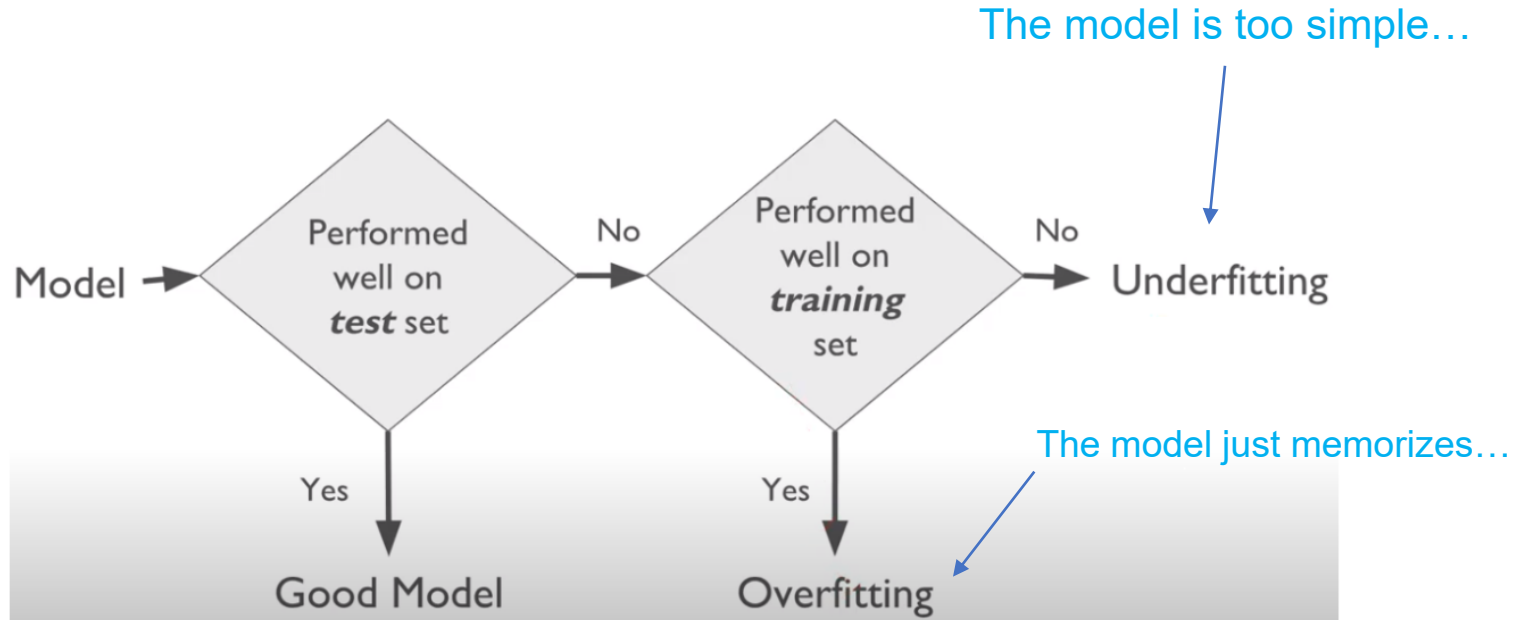
Python package:

```
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(my_model, X, y, cv = 5)
```

Model Evaluation

- Goal: Minimize error on the **test** data
- Error on the **training** data may not be a good indicator of performance on future data
 - New data will not be exactly the same as the training data!
 - **Overfitting** – fitting the training data too precisely leads to learning too much peculiarities in training data and generates poor results on new data.
- Model evaluation should be interwoven seamlessly into the methodology for building and evaluating a data model on each of the training set and the test set

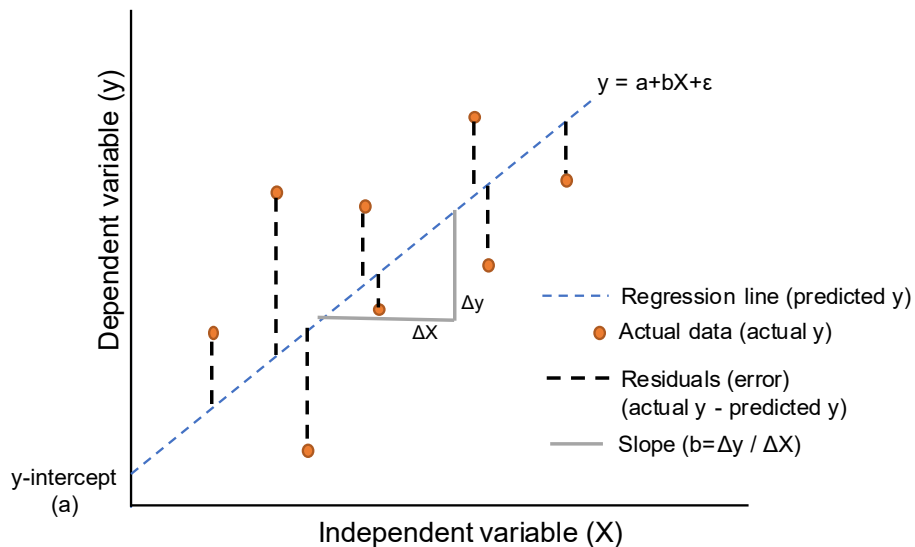
Model Evaluation



Model Performance for Numerical Prediction (Regression)

- Not the same as “goodness-of-fit”
 - We want to know how well the model predicts new data, not how well it fits the data it was trained with
- Key component of most measures is difference between actual y and predicted y (“error,” “loss”)
- Some common measures used:
 - **MAE** or MAD: Mean absolute error (deviation) - Gives an idea of the magnitude of errors
 - Average error - Gives an idea of systematic over- or under-prediction
 - MAPE: Mean absolute percentage error
 - **RMSE** (root-mean-squared-error): Square the errors, find their average, take the square root
 - Total SSE: Total sum of squared errors
 - **R-squared**

Example – Linear Regression



$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}|$$

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y})^2}{\sum (y_i - \bar{y})^2}$$

Where,

$\hat{y}$ – predicted value of y

$\bar{y}$ – mean value of y

Model Performance for Binary Classification

y_true
(20% of **y**)

y_pred
(y_pred computed using 20% of **x**)

0	0
1	0
1	1
0	1
1	1
0	0
1	1
1	0
1	1
0	1
1	1
0	0
0	0
1	0
1	0

Main idea: **count!**

- # of items that were correctly predicted
 - Number of 1's (people that have diabetes) you correctly predicted as 1's (aka, true positives)
 - Number of 0's (people that don't have diabetes) you correctly predicted as 0's (aka, true negatives)
- # of items that were incorrectly predicted
 - Number of 1's you predicted as 0's (false negatives)
 - Number of 0's you predicted as 1's (false positives)

Model Performance for Binary Classification

y_true
(20% of **y**)

y_pred
(y_pred computed using 20% of **X**)

0	0
1	0
1	1
0	1
1	1
0	0
1	1
1	0
1	1
0	1
1	1
0	0
0	0
1	0
1	0

Confusion matrix

		Predicted Condition	
		Pred. Negative	Pred. Positive
Actual Condition	Negative (N)	true negative (TN)	false positive (FP) type 1 error
	Positive (P)	false negative (FN) type 2 error	true positive (TP)

Example: true positives

y_true
(20% of **y**)

y_pred
(y_pred computed using 20% of **x**)

0	0
1	0
1	1
0	1
1	1
0	0
1	1
1	0
1	1
0	1
1	0
1	0
0	0
0	0
1	0
1	0

		Predicted Condition	
		Pred. Negative	Pred. Positive
Actual Condition	Negative (N)	true negative (TN)	false positive (FP) type 1 error
	Positive (P)	false negative (FN) type 2 error	true positive (TP)

		Predicted Condition	
		0	1
Actual Condition	0		
	1		5

Full Confusion Matrix

y_true
(20% of y)

y_pred
(y_pred computed using 20% of X)

0	0
1	0
1	1
0	1
1	1
0	0
1	1
1	0
1	1
0	1
1	1
0	0
0	0
1	0
1	0

Actual
Condition

Predicted Condition

		Predicted Condition	
		Pred. Negative	Pred. Positive
Actual Condition	Negative (N)	true negative (TN)	false positive (FP) type 1 error
	Positive (P)	false negative (FN) type 2 error	true positive (TP)

Predicted Condition

Actual
Condition

		Predicted Condition	
		0	1
Actual Condition	0	4	2
	1	4	5

Confusion Matrix - vocabulary

Some common measures:

- **(actual)P** = FN + TP = 4 + 5 = 9
- **(actual)N** = TN + FP = 4 + 2 = 6
- **Accuracy** = (TP+TN)/total = 9/15 = 0.6
- **Precision** = TP/(TP+FP) = 5/7 = 0.71
- **Recall/Sensitivity/TPR** = TP/P = 5/9 = 0.55
- **Fall-out/FPR** = FP/N = 2/6 = 0.33
- **F1 score** (harmonic mean, in [0,1])

$$F1 = 2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall}) = 0.62$$

		Predicted Condition	
		Negative	Positive
Actual Condition	Negative (N)	true negative (TN)	false positive (FP)
	Positive (P)	false negative (FN)	true positive (TP)

		Predicted Condition	
		0	1
Actual Condition	0	4	2
	1	4	5

Metric Cheat Sheet

Metric	Formula	Easy Analogy	Use When...
Accuracy	$TP + TN / \text{Total}$	Overall correctness (like grading an exam)	False positives & false negatives are equally bad
Precision	$TP / (TP + FP)$	Spam filter: If low, marks important emails as spam	False positives are worse than false negatives
Recall (Sensitivity, TPR)	TP / P	Metal detector: If low, lets weapons through	False negatives are worse than false positives
Specificity	$TN / (TN + FP)$	Fire alarm: If low, it goes off for burnt toast	Avoiding false positives is critical
False Positive Rate (FPR)	FP / N	False alarms: Measures how often a negative is misclassified	Lower is better for fraud detection or medical tests
F1-Score	$2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$	Soccer goalie: Balances precision & recall	Balance precision & recall (e.g., fraud detection)
MCC	Mixes everything	Chess rating: Measures overall performance	Best when data is imbalanced

Probabilities and Cutoffs for Classification

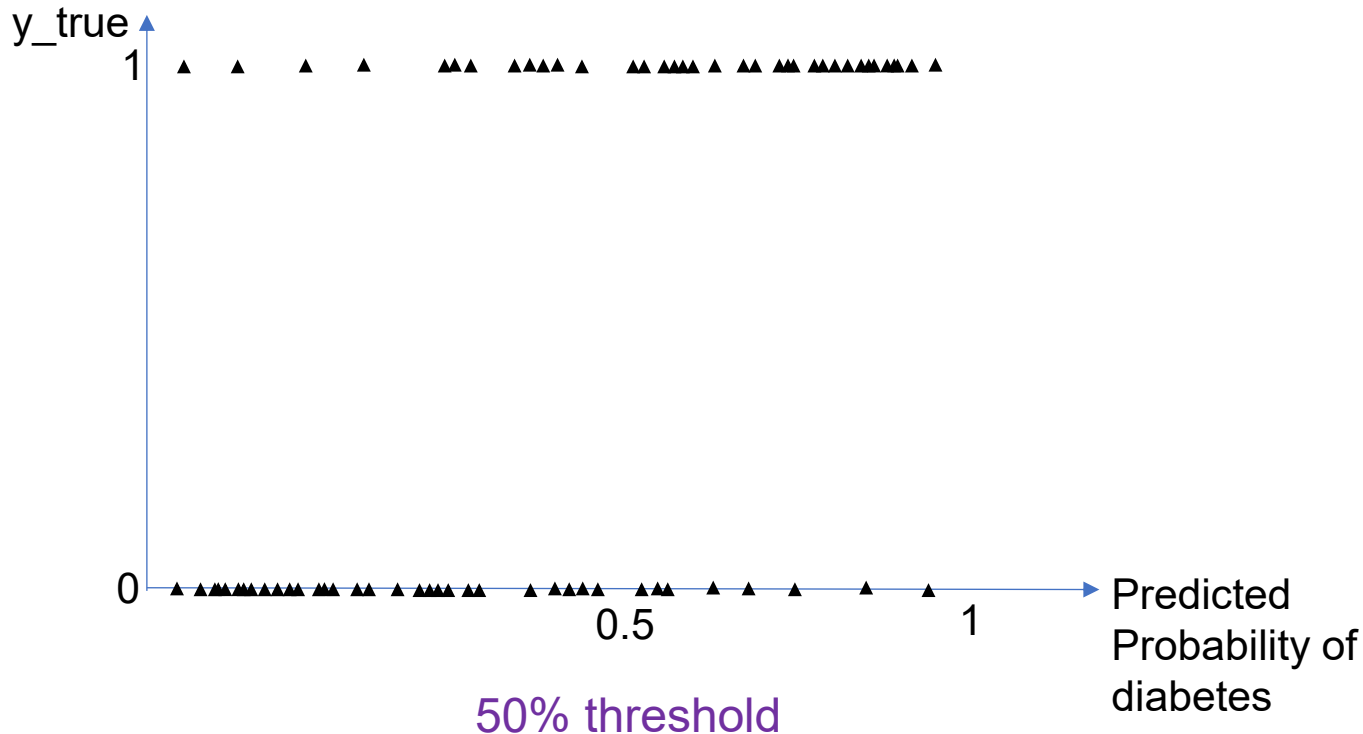
- Most algorithms classify via a 2-step process. For each record:
 - Compute probability of belonging to class “1”
 - Compare to cutoff value, and classify accordingly as “1” or “0”
- Default cutoff value is 0.50
 - If ≥ 0.50 , classify as “1”
 - If < 0.50 , classify as “0”
- But this is not necessarily the best one
 - In fact, it's almost never the best one!
- [Demo](#)

Example: probability of diabetes – 50% threshold

y_true
(20% of y)

pred. prob.
20% of x

0	.20
1	.45
1	.83
0	.73
1	.62
0	.43
1	.92
1	.40
1	.75
0	.55
1	.65
0	.34
0	.33
1	.21
1	.33
...	...

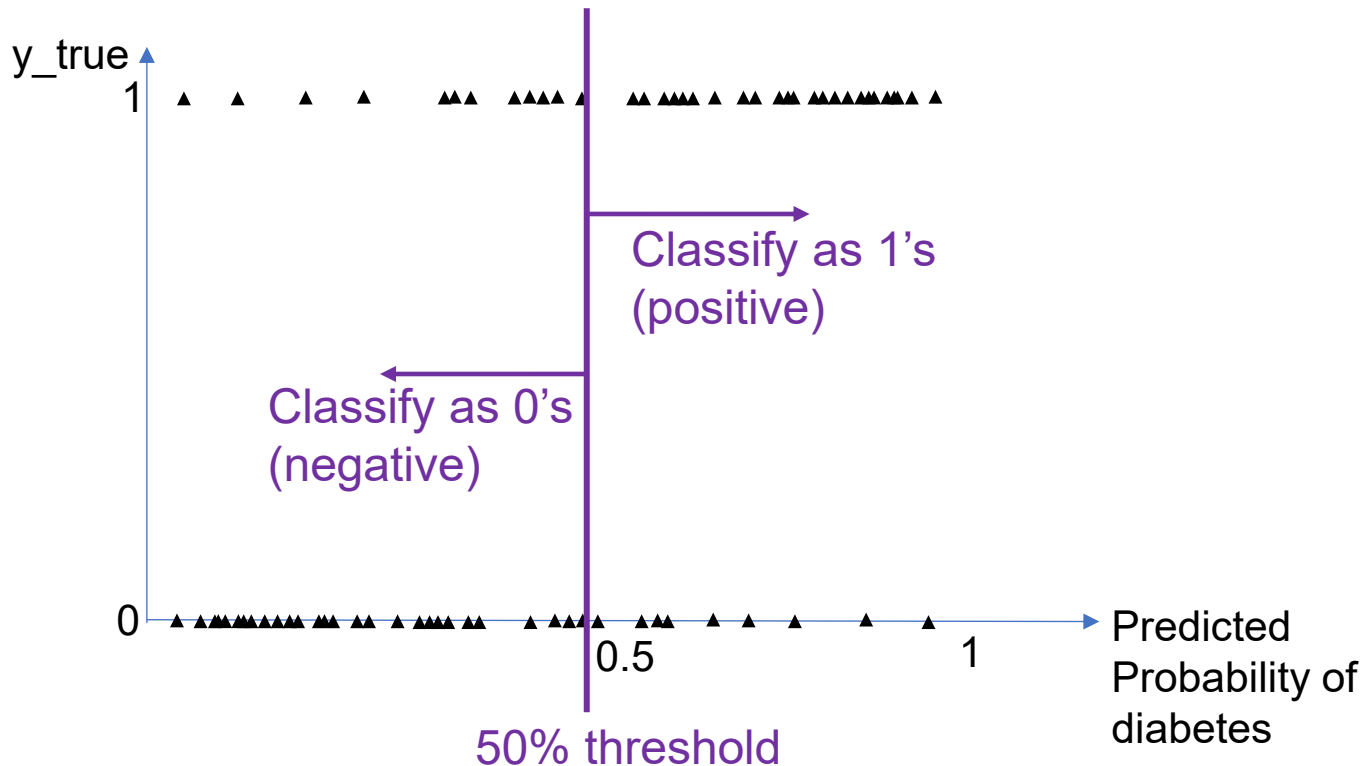


Example: probability of diabetes – 50% threshold

y_true
(20% of y)

pred. prob.
20% of x

0	.20
1	.45
1	.83
0	.73
1	.62
0	.43
1	.92
1	.40
1	.75
0	.55
1	.65
0	.34
0	.33
1	.21
1	.33
...	...

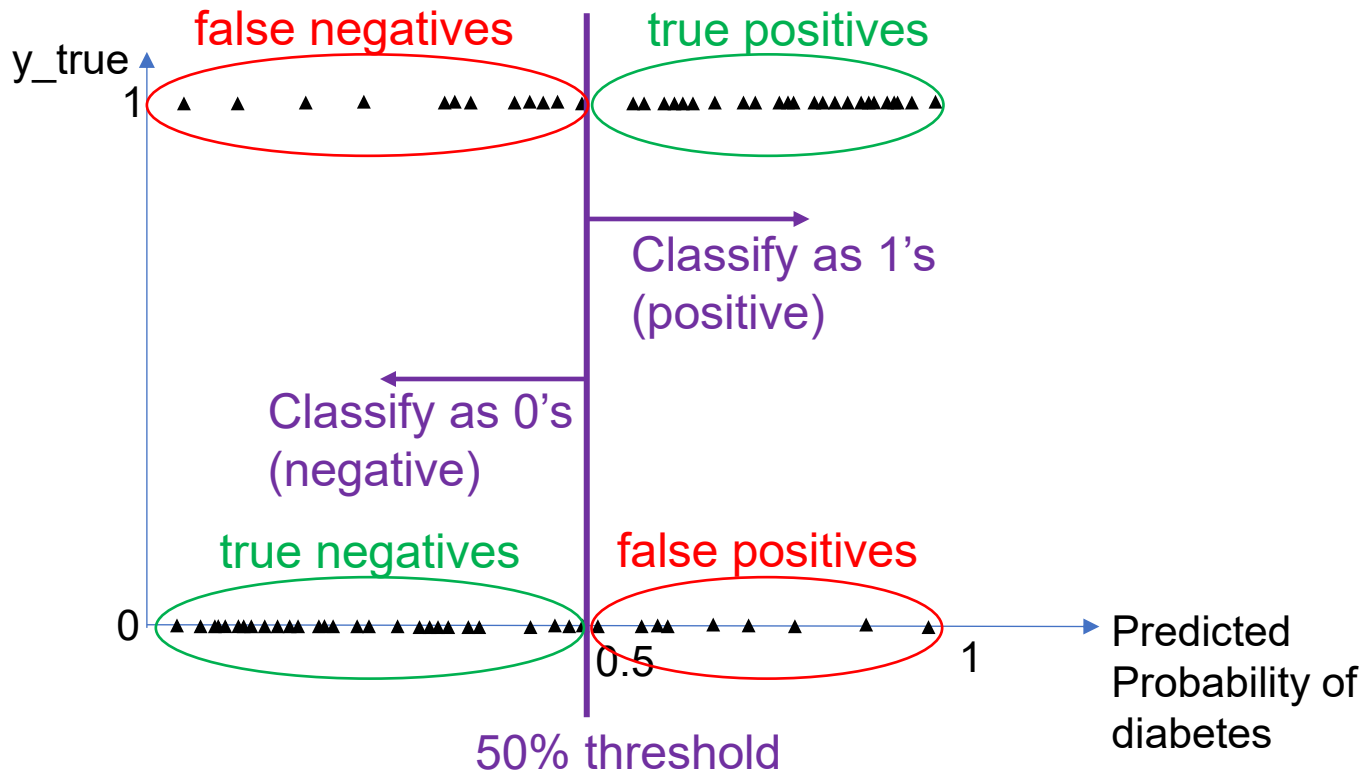


Example: probability of diabetes – 50% threshold

y_true
(20% of y)

pred. prob.
20% of x

0	.20
1	.45
1	.83
0	.73
1	.62
0	.43
1	.92
1	.40
1	.75
0	.55
1	.65
0	.34
0	.33
1	.21
1	.33
...	...

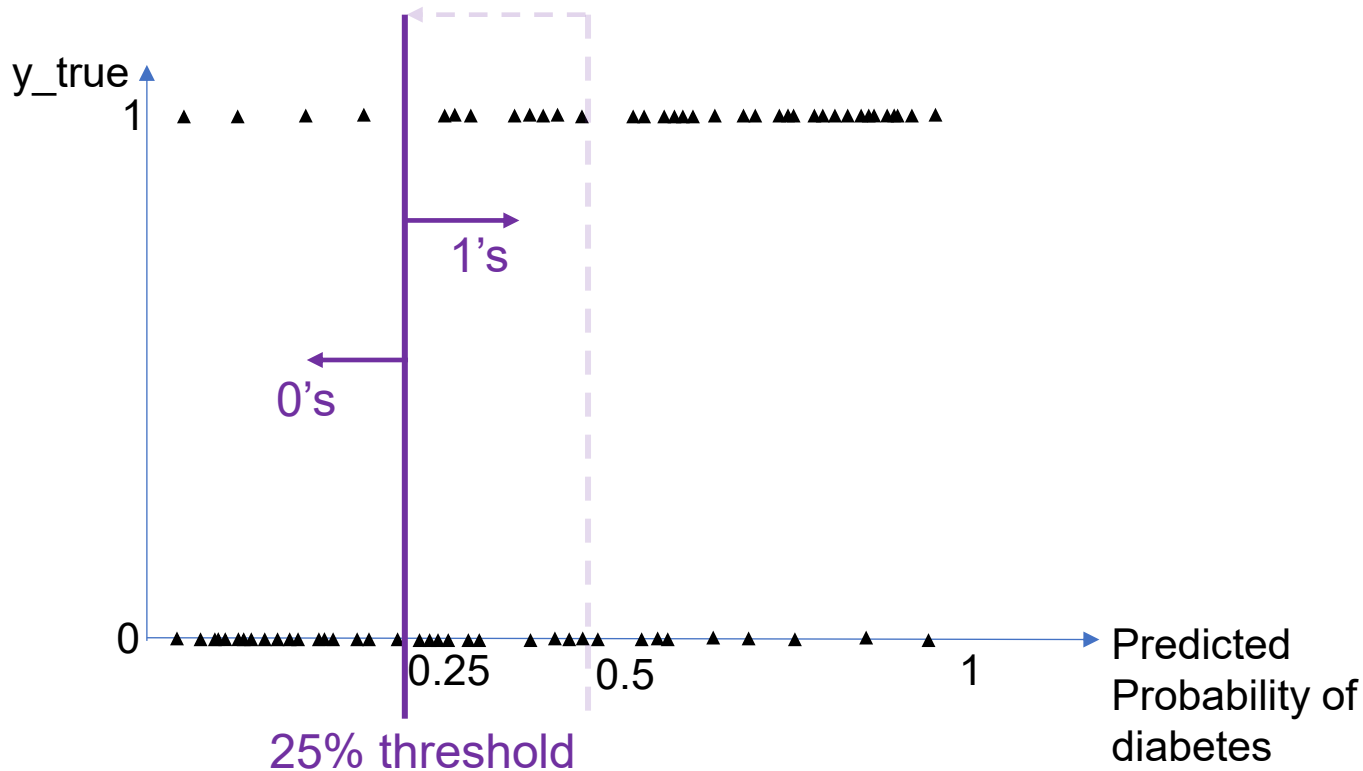


Example: probability of diabetes – 25% threshold

y_true
(20% of y)

pred. prob.
20% of x

0	.20
1	.45
1	.83
0	.73
1	.62
0	.43
1	.92
1	.40
1	.75
0	.55
1	.65
0	.34
0	.33
1	.21
1	.33
...	...

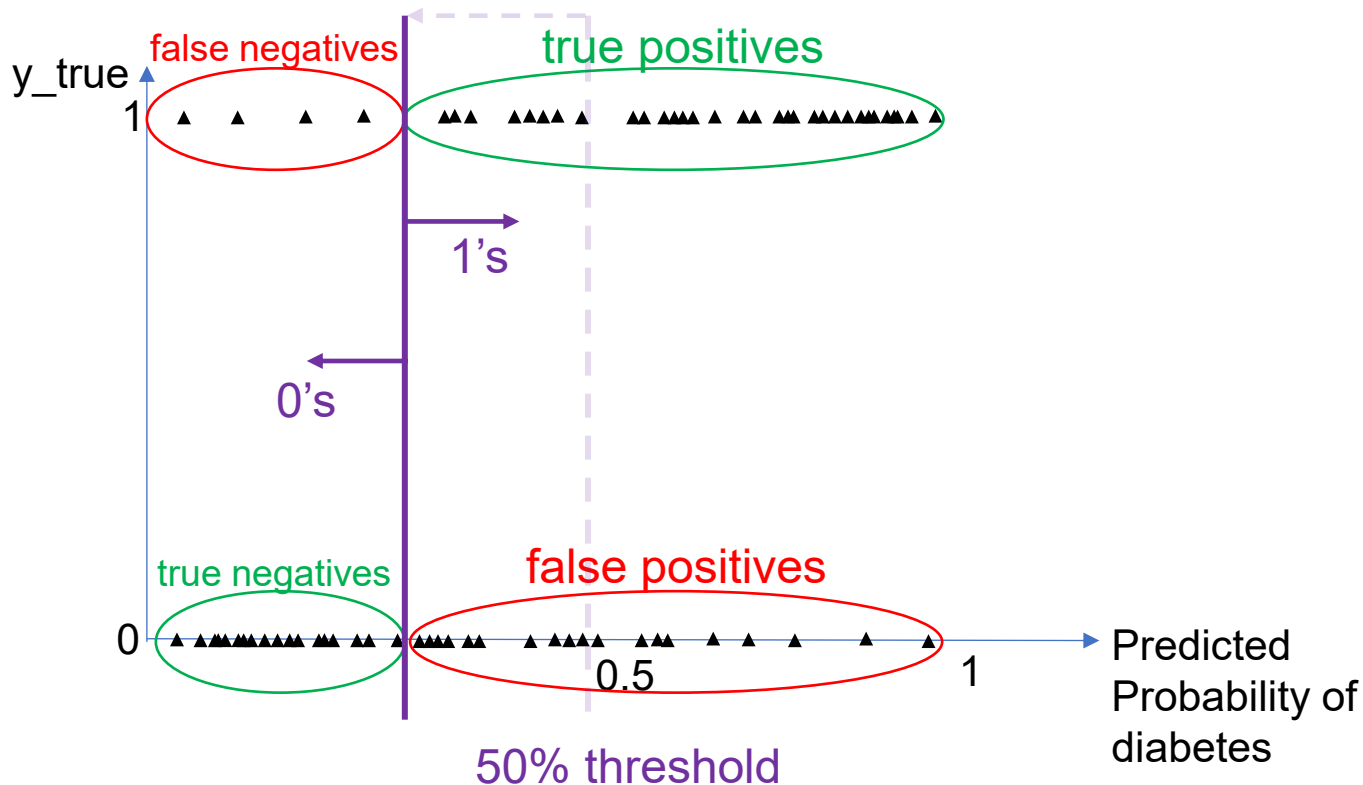


Example: probability of diabetes – 25% threshold

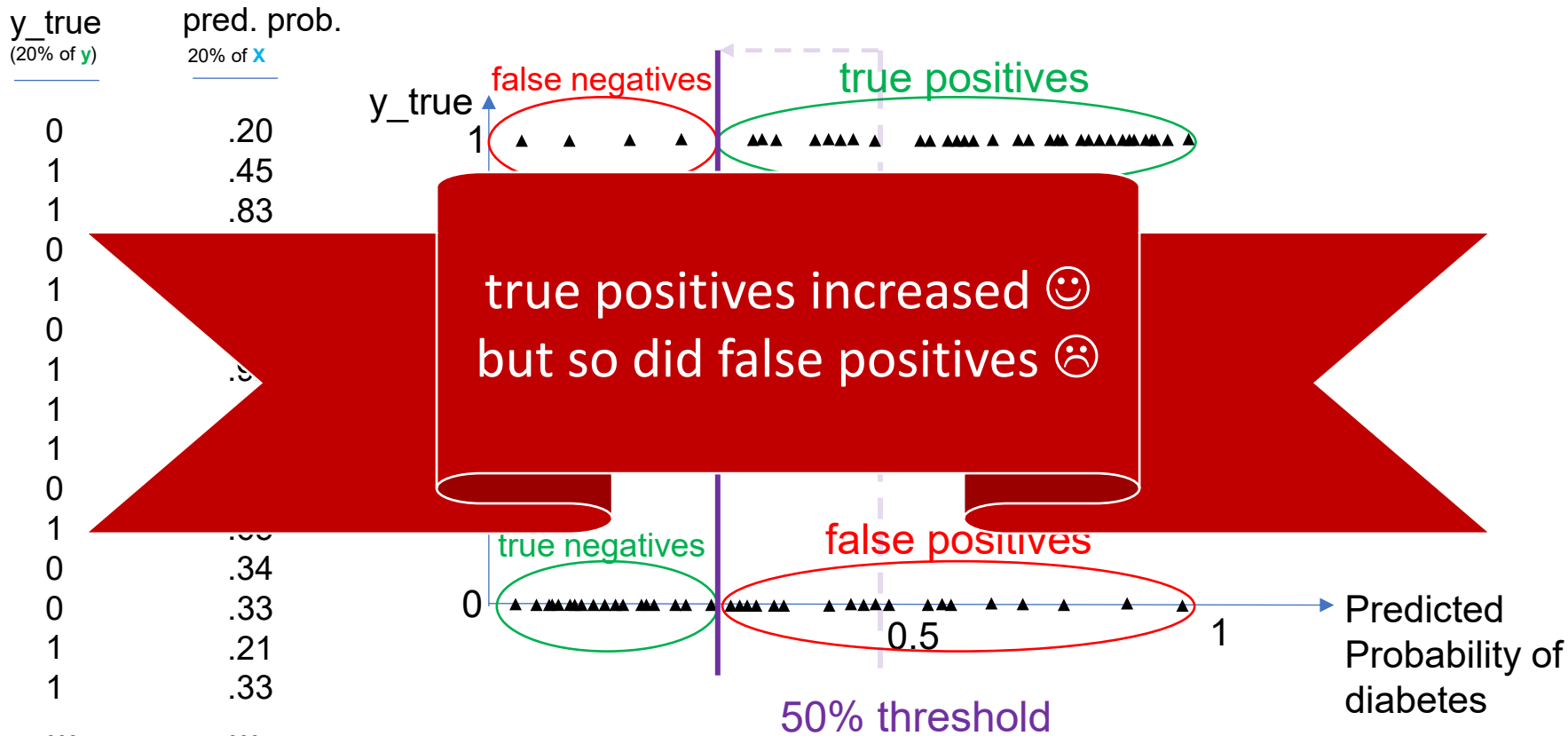
y_true
(20% of y)

pred. prob.
20% of x

0	.20
1	.45
1	.83
0	.73
1	.62
0	.43
1	.92
1	.40
1	.75
0	.55
1	.65
0	.34
0	.33
1	.21
1	.33
...	...



Example: probability of diabetes – 25% threshold



Each cutoff leads to a different Confusion matrix!

Predicted Probability of having diabetes

	actual	pred.
	1	0.9959
	1	0.9875
	1	0.9844
	1	0.9804
	1	0.9481
	1	0.8892
	1	0.8476
if cutoff is 0.75, then 8 records classified as "1"	0	0.7628
	1	0.7069
	1	0.6807
	1	0.6563
	0	0.6224

	actual	pred.
	1	0.5055
	0	0.4713
	0	0.3371
	1	0.2179
	0	0.1992
	0	0.1494
	0	0.0479
if cutoff is 0.25, then 11 records classified as "1"	0	0.0383
	0	0.0248
	0	0.0218
	0	0.0161
	0	0.0031

Cutoff **0.75** (Accuracy 0.7500)
 TPR: 7/12, FPR: 1/12

	Prediction	
Actual	0	1
0	11	1
1	5	7

Cutoff = **0.25** (Accuracy .66)
 TPR: 12/12, FPR: 8/12

	Prediction	
Actual	0	1
0	4	8
1	0	12

Each cutoff leads to a different Confusion matrix!

Predicted Probability of having diabetes

actual	pred.	actual	pred.
1	0.9959	1	0.5055
1	0.9875	0	0.4713
1	0.9844	0	0.3371
1			
1			

if cutoff is
records clas

How can we visualize the
trade-offs?

C

TPR: 7/12, FPR: 1/12

Cutoff

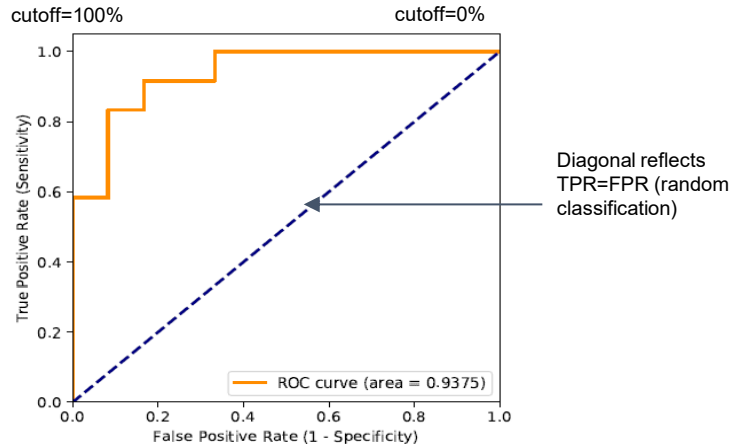
TPR: 12/12, FPR: 8/12

	Prediction	
Actual	0	1
0	11	1
1	5	7

	Prediction	
Actual	0	1
0	4	8
1	0	12

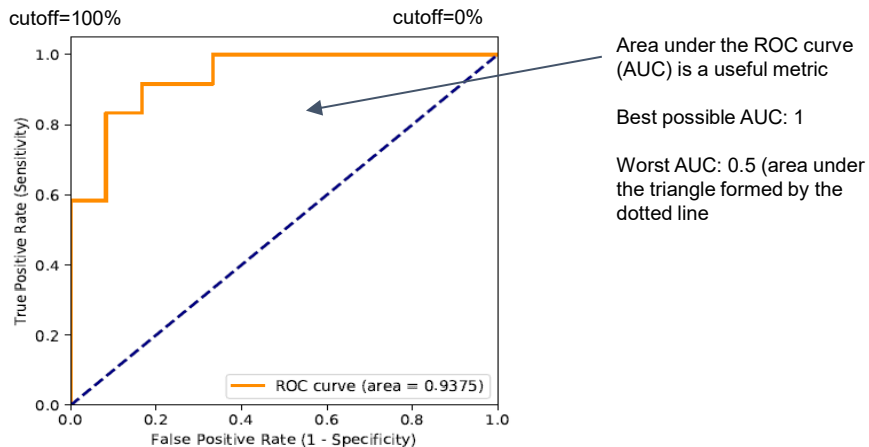
Receiver Operating Characteristics Curve (ROC)

- ROC curve plots TPR (on the y-axis) against FPR (on the x-axis) at all possible cutoff values



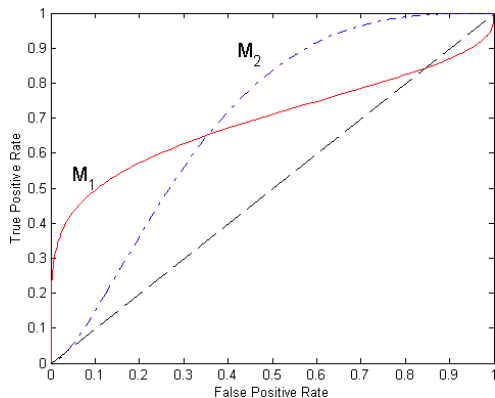
Receiver Operating Characteristics Curve (ROC)

- Area under the curve “AUC” often used to compare different models



Receiver Operating Characteristics (ROC)

- ROC curves also useful to compare different predictive models, M1, M2, etc.



Here, no model consistently outperforms the other

- M1 is better for small FPR
- M2 is better for large FPR

When One cell in the matrix is more Important...

- In many cases it is more important to identify members of one class
 - Tax fraud
 - Credit default
 - Response to promotional offer
 - Detecting electronic network intrusion
 - Predicting delayed flights
- In such cases, we are willing to tolerate greater overall error, in return for better identifying the important class for further attention
- Example: For predicting loan default, it is more important to correct identify a real default. It would be better to have high TPR (predicted defaults over total real defaults) than high precision, which takes account of incorrectly predicted defaults (false positives).

Solution: cost-benefit matrix!

		Predicted Condition	
		0	1
Actual Condition	0	4	2
	1	4	5

confusion matrix
for a given
cutoff value (say 50%)

		Predicted Condition	
		0	1
Actual Condition	0	benefit: \$100K	cost: -\$30K
	1	cost: -\$70K	benefit: \$50K

cost-benefit matrix
(on a per person basis,
does not change
with cutoff)

Total cost associated with the current cutoff

		Predicted Condition	
		0	1
Actual Condition	0	4	2
	1	4	5

$$4 \cdot 100 + 5 \cdot 50 - 2 \cdot 30 - 4 \cdot 70 = \$310K$$

This is the net benefit for Cutoff 50%.

		Predicted Condition	
		0	1
Actual Condition	0	benefit: \$100k	cost: -\$30k
	1	cost: -\$70k	benefit: \$50k

Repeat for all other cutoffs.

The best cutoff maximizes net benefit

